

Lecture 13: Apache Spark's Structured APIs

DSL351 / DSP352: Big Data Analytics

Amit Kumar Dhar

ED1, 406A, amitkdhar@iitbhlai

February 9, 2026

Introduction: The Move to Structure

Context: The Spark Evolution

- **Spark 1.x:** RDDs were the primary API. Powerful but opaque.
- **Spark 2.x:** Introduction of Structured APIs (DataFrames, Datasets, SQL) as the standard.
- **Spark 3.x:** Continued improvements, Adaptive Query Execution (AQE).

*Goal: To tell Spark **what** to do, not **how** to do it.*

Recap: The RDD Abstraction

The **Resilient Distributed Dataset (RDD)** is the most basic abstraction.

- **Resilient:** Fault-tolerant (recomputes missing partitions).
- **Distributed:** Data resides on multiple nodes.
- **Dataset:** A collection of partitioned data.

Recap: RDD Internals

Three vital characteristics of an RDD:

1. **Dependencies:** A list of parent RDDs (Lineage).
2. **Partitions:** Atomic chunks of data.
3. **Compute Function:** `Partition => Iterator[T]`.

The Problem with RDDs: Opacity

To Spark, an RDD is a collection of opaque Java/Scala objects.

- Spark doesn't know if the object is a Person, a Log, or a Graph.
- It doesn't know the fields inside the object.
- It cannot optimize data storage (serialization).

The Problem with RDDs: No Optimization

When you use RDD transformations (map, filter), you are passing **Lambda Functions**.

- Spark cannot inspect the code inside the lambda.
- Spark cannot "rearrange" your operations for efficiency.
- Spark executes exactly what you tell it, even if it's inefficient.

Example: Aggregating Ages with RDDs

```
1 // Create an RDD of tuples (name, age)
2 val dataRDD = sc.parallelize(Seq(
3   ("Brooke", 20), ("Denny", 31), ("Jules", 30), ("TD", 35), ("Brooke", 25)
4 ))
5
6 // Use map and reduceByKey transformations
7 val agesRDD = dataRDD
8   .map(x => (x._1, (x._2, 1)))
9   .reduceByKey((x, y) => (x._1 + y._1, x._2 + y._2))
10  .map(x => (x._1, x._2._1 / x._2._2))
11
12 agesRDD.collect().foreach(println)
```

Critique: Cryptic, low-level, hard to read, opaque to optimization.

Example: Aggregating Ages with RDDs

```
1 // Create an RDD of tuples (name, age)
2 val dataRDD = sc.parallelize(Seq(
3   ("Brooke", 20), ("Denny", 31), ("Jules", 30), ("TD", 35), ("Brooke", 25)
4 ))
5
6 // Use map and reduceByKey transformations
7 val agesRDD = dataRDD
8   .map(x => (x._1, (x._2, 1))) // Map to (name, (age, 1))
9   .reduceByKey((x, y) => (x._1 + y._1, x._2 + y._2)) // Sum ages and counts
10  .map(x => (x._1, x._2._1 / x._2._2)) // Average: total_age / count
11
12 agesRDD.collect().foreach(println)
```

Critique: Cryptic, low-level, hard to read, opaque to optimization.

Example: Aggregating Ages with DataFrames

Task: Same task, but using the DataFrame API.

```
1 import org.apache.spark.sql.functions.avg
2
3 // Create a DataFrame
4 val dataDF = spark.createDataFrame(Seq(
5   ("Brooke", 20), ("Denny", 31), ("Jules", 30), ("TD", 35), ("Brooke", 25)
6 )).toDF("name", "age")
7
8 // Group by name and compute average age
9 val avgDF = dataDF.groupBy("name").agg(avg("age"))
10
11 avgDF.show()
```

Benefit: Expressive, high-level, easy to read, optimized by Catalyst.

Motivation for Structure

Structure yields key benefits:

1. **Expressivity**: Code reads like a high-level query.
2. **Simplicity**: Common patterns (Aggregation, Filtering) are built-in.
3. **Composability**: Easy to chain operations.
4. **Uniformity**: Same API across languages (Scala, Python, Java, R).

The DataFrame API

What is a DataFrame?

A **DataFrame** is a distributed collection of data organized into named columns.

- Conceptually equivalent to a table in a relational database.
- Or a data frame in R/Python (Pandas).
- Under the hood: It is a Dataset [Row].

Key Properties of DataFrames

- **Immutable:** You cannot modify a DataFrame; you transform it into a new one.
- **Lazy Evaluation:** Computation happens only when an *Action* is triggered.
- **Distributed:** Rows are partitioned across the cluster.
- **Schema-aware:** It knows the data types of its columns.

Spark has its own internal type system (Catalyst types) that maps to language-specific types.

This allows Spark to maintain accuracy and optimization regardless of whether you use Python or Scala.

Basic Scala Types in Spark

Spark Type	Scala Type
ByteType	Byte
ShortType	Short
IntegerType	Int
LongType	Long
FloatType	Float
DoubleType	Double
StringType	String
BooleanType	Boolean
DecimalType	java.math.BigDecimal

Complex Data Types

Spark supports complex, nested data structures:

- **ArrayType(elementType, containsNull)**: Sequence of elements.
- **MapType(keyType, valueType, valueContainsNull)**: Key-value pairs.
- **StructType(fields)**: Represents a Row or a nested object.
- **StructField(name, dataType, nullable)**: A field within a StructType.

A **Schema** defines the structure of the DataFrame:

- Column Names
- Data Types
- Nullability (Can the column contain nulls?)

Why Define Schemas?

Schema-on-Read (Inferring schema) is convenient but costly.

Defining Schema Upfront:

- Relieves Spark from scanning the file to infer types.
- Prevents Spark from creating a separate job just to read the schema.
- Detects data errors early (if data doesn't match schema).
- Essential for production pipelines.

Defining Schema: Programmatic Approach

Using StructType and StructField:

```
1 import org.apache.spark.sql.types._
2
3 val schema = StructType(Array(
4     StructField("author", StringType, false),
5     StructField("title", StringType, false),
6     StructField("pages", IntegerType, false)
7 ))
```

Defining Schema: DDL Approach

Using Data Definition Language (DDL) strings. simpler and concise.

```
1 // Simple DDL
2 val schemaDDL = "author STRING, title STRING, pages INT"
3
4 // Complex DDL
5 val complexSchema = "'Id' INT, 'Date' STRING, 'Hits' ARRAY<INT>"
```

A **Column** is an object representing a column in a DataFrame.

- It supports operations like arithmetic, logical comparisons, and string manipulations.
- Class: `org.apache.spark.sql.Column`

Accessing Columns

Three ways to refer to a column in Scala:

```
1 import org.apache.spark.sql.functions._
2
3 // 1. Using col() function
4 df.select(col("author"))
5
6 // 2. Using $ notation (Scala specific helper)
7 df.select($"author")
8
9 // 3. Using SQL expression string
10 df.selectExpr("author")
```

Expressions allow you to use SQL-like syntax within DataFrame operations.

```
1 // Arithmetic expression on a column
2 df.selectExpr("pages * 2 as double_pages")
3
4 // Boolean expression
5 df.filter("pages > 200")
```


A **Row** is a generic object representing a record in the DataFrame.

- It is an ordered collection of fields.
- In Scala, it is untyped (access by index).
- Class: `org.apache.spark.sql.Row`

Creating and Accessing Rows

```
1 import org.apache.spark.sql.Row
2
3 // Create a Row
4 val myRow = Row("Learning Spark", 350, true)
5
6 // Access Fields (Zero-indexed)
7 val title = myRow.getString(0)
8 val pages = myRow.getInt(1)
9 // val pages = myRow(1) // Returns Any
```

Creating DataFrames

The entry point to programming Spark with the Dataset and DataFrame API.

- Unified entry point (replaces `SparkContext`, `SQLContext`, `HiveContext`).
- Available as the variable `spark` in the shell.

Interface to load data from external storage systems.

Structure: `spark.read.format(...).option(...).schema(...).load(...)`

Reading CSV

```
1 val fireDF = spark.read
2   .format("csv")
3   .option("header", "true")
4   .option("inferSchema", "true") // Not recommended for prod
5   .load("/data/fire_calls.csv")
```

Reading JSON

```
1 val blogsDF = spark.read
2   .format("json")
3   .schema(schema) // Using defined schema
4   .load("/data/blogs.json")
```

Parquet is the default format for Spark. It stores schema in the file footer.

```
1 val usersDF = spark.read.load("/data/users.parquet")  
2 // OR  
3 val usersDF = spark.read.parquet("/data/users.parquet")
```


Interface to save data to external storage.

Structure: `df.write.format(...).option(...).mode(...).save(...)`

Scala/Java	Behavior
<code>SaveMode.ErrorIfExists</code>	Throw exception if exists (Default).
<code>SaveMode.Append</code>	Append data to existing files.
<code>SaveMode.Overwrite</code>	Delete and rewrite.
<code>SaveMode.Ignore</code>	Do nothing if exists.

Common DataFrame Operations

Projections (Select)

Selecting a subset of columns.

```
1 val subDF = fireDF.select("IncidentNumber", "CallType")
2
3 // Select and rename
4 fireDF.select($"IncidentNumber", $"CallType".alias("Type"))
```

Filtering (Where/Filter)

Filtering rows based on a condition.

```
1 // Using Column expression
2 fireDF.filter($"CallType" === "Medical Incident")
3
4 // Using SQL string
5 fireDF.where("CallType = 'Medical Incident'")
6
7 // Multiple conditions
8 fireDF.filter($"CallType".isNotNull && $"Delay" > 5)
```

Renaming Columns

`withColumnRenamed` returns a new `DataFrame` with the column renamed.

```
1 val renamedDF = fireDF
2   .withColumnRenamed("Delay", "ResponseTime")
```

Adding Columns

`withColumn` adds a new column or replaces an existing one.

```
1 // Create a new column based on existing ones
2 val withRatioDF = fireDF
3   .withColumn("DelayRatio", $"Delay" / 100)
4
5 // Constant column
6 import org.apache.spark.sql.functions.lit
7 fireDF.withColumn("Source", lit("SF_Fire_Dept"))
```

Dropping Columns

Removing columns that are not needed.

```
1 val cleanDF = fireDF.drop("Delay", "WatchDate")
```


Handling Nulls

Spark provides methods to handle missing data via DataFrameNaFunctions.

```
1 // Drop rows with any nulls
2 df.na.drop()
3
4 // Fill nulls with a value
5 df.na.fill(0, Seq("Delay"))
```

Date and Time Functions

org.apache.spark.sql.functions contains standard date helpers.

```
1 val datesDF = fireDF
2   .withColumn("IncidentDate", to_timestamp($"CallDate", "MM/dd/yyyy"))
3   .withColumn("Year", year($"IncidentDate"))
4   .withColumn("Month", month($"IncidentDate"))
```

Aggregations: Concept

Aggregations are usually performed in two steps:

1. **Group:** `groupBy("column")` - Returns `RelationalGroupedDataset`.
2. **Aggregate:** `count()`, `sum()`, `avg()`, etc.

Aggregations: Example

Question: How many calls per CallType?

```
1 fireDF
2   .groupBy("CallType")
3   .count()
4   .orderBy($"count".desc)
5   .show(5)
```

Aggregations: Statistical Stats

Computing summary statistics.

```
1 fireDF.select(  
2     sum("NumAlarms"),  
3     avg("Delay"),  
4     min("Delay"),  
5     max("Delay")  
6 ).show()
```

The Dataset API

What is a Dataset?

- Introduced in Spark 1.6, unified in Spark 2.0.
- **Strongly Typed:** `Dataset[T]`.
- **Domain Specific Objects:** T can be a custom class.
- Available in Scala and Java.

Typed vs Untyped API

- **DataFrame:** `Dataset[Row]` (Untyped View).
- **Dataset:** `Dataset[T]` (Typed View).

Since Python/R are dynamically typed, they primarily use DataFrames. Scala/Java can use both.

Defining a Case Class

To create a Dataset in Scala, we define the schema as a **Case Class**.

```
1 case class IoTData (  
2     device_id: Long,  
3     ip: String,  
4     temp: Long,  
5     humidity: Long  
6 )
```

Creating a Dataset

Use the `.as[T]` encoder to convert DataFrame to Dataset.

```
1 val ds = spark.read
2   .json("iot_devices.json")
3   .as[IoTData]
4
5 ds.show()
```

Functional Transformations

Datasets support functional programming methods (map, filter, reduce) using lambdas.

```
1 // Using a lambda function
2 val hotSensors = ds.filter(d => d.temp > 30)
3
4 // Map operation
5 val deviceNames = ds.map(d => d.device_id)
```

DataFrame (Runtime Error):

```
1 // "tempp" is a typo
2 df.filter("tempp > 30")
3 // Compiles, but crashes at runtime!
```

Dataset (Compile Error):

```
1 // "tempp" is a typo
2 ds.filter(d => d.tempp > 30)
3 // Fails to compile!
4 // "value tempp is not a member of
   IoTData"
```

DataFrames vs Datasets

Feature	DataFrame	Dataset
Language	All	Scala/Java
Type Safety	Runtime	Compile Time
Data Type	Row	Case Class / Bean
Optimization	Catalyst	Catalyst + Encoders
Readability	DSL	Functional + DSL

When to use RDDs?

RDDs are still the foundation, but use them only when:

1. You need fine-grained control over physical data layout.
2. You are manipulating raw unstructured data that doesn't fit a schema.
3. You need to use a legacy library that only accepts RDDs.

Under the Hood: The Spark SQL Engine

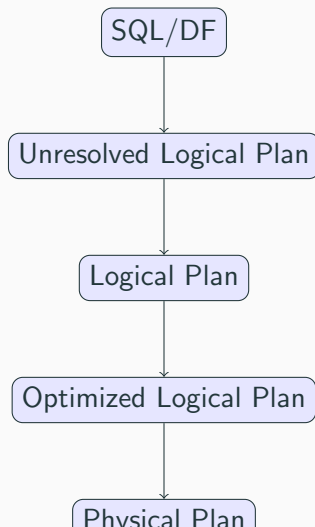
The high-level APIs (DF/DS/SQL) are powered by the **Spark SQL Engine**.

Two main components:

1. **Catalyst Optimizer**: For query planning.
2. **Project Tungsten**: For execution optimization.

The Catalyst Optimizer

Takes a logical plan and transforms it into an efficient physical plan.



Phase 1: Analysis

Spark consults an internal **Catalog** (metadata) to verify:

- Do the table names exist?
- Do the column names exist in those tables?
- Are the data types valid for the operations (e.g., can't divide a String)?

Phase 2: Logical Optimization

Applies standard rule-based optimizations:

- **Constant Folding:** $100 + 20 \rightarrow 120$.
- **Predicate Pushdown:** Pushing filters down to the data source.
- **Projection Pruning:** Removing unused columns early.
- **Boolean Simplification.**

Phase 3: Physical Planning

Generates one or more physical plans and selects the best one using a **Cost-Based Optimizer (CBO)**.

Example decisions:

- Should I use a Broadcast Join or a Sort Merge Join?
- Which partitions should I read?

Phase 4: Code Generation

Once the physical plan is selected, Spark generates Java Bytecode.

Whole-Stage Code Generation (Project Tungsten):

- Collapses the entire query tree into a single function.
- Eliminates expensive virtual function calls.
- Keeps data in CPU registers (L1/L2 cache) for speed.

Inspecting the Plan

You can see this process using the `explain()` method.

```
1 df.explain(true)
```

- Parsed Logical Plan
- Analyzed Logical Plan
- Optimized Logical Plan
- Physical Plan

- **Structure** is the key to performance and usability in Spark 2.x+.
- **DataFrames** are untyped tables, easy to use, and optimized.
- **Datasets** provide type safety for engineering tasks in Scala/Java.
- Always define **Schemas** for production code.
- The **Catalyst Optimizer** ensures your high-level code runs efficiently.

- *Learning Spark, 2nd Edition* (O'Reilly) - Chapter 3.
- Apache Spark Documentation:
<https://spark.apache.org/docs/latest/sql-programming-guide.html>